Examinationsuppgift 1

Komplettering - Hunddagiset

Du tar över ett befintligt C#-projekt för ett hunddagis. Systemet är delvis färdigbyggt men innehåller fel, ofullständig funktionalitet och bristande felhantering.

Din uppgift är att självständigt felsöka, korrigera och färdigställa konsolappen så att den uppfyller följande krav:

- Projektet ska kunna byggas och startas utan kompileringsfel.
- Menyn ska innehålla: 1. Visa hundar, 2. Visa lediga platser, 3. Checka in hund, 4. Checka ut hund samt 0. Avsluta.
- Vid varje programstart ska en liten, en medelstor och en stor hund vara incheckade. Varje hund ska ha ett unikt id, ett namn och en storlek.
- Visa hundar ska visa hundarnas id, namn och storlek. Platsåtgången per hund ska inte visas. Sist i listan ska dagisets belastning visas i procent.
- Visa lediga platser ska visa använd kapacitet, maxkapacitet och ledig kapacitet. Hela värden ska visas utan onödig decimal, exempelvis 7 i stället för 7,0.
- Det ska gå att checka in en hund med namn och storlek samt checka ut en hund med hjälp av dess id.
- En incheckning som skulle överskrida maxkapaciteten ska avvisas. En hund som fyller kapaciteten exakt ska kunna checkas in. Utcheckning ska frigöra rätt mängd kapacitet.
- Felaktig eller tom användarinmatning och id:n som inte finns ska hanteras utan att programmet kraschar eller ändrar registrerade hundar.
- Koden ska vara uppdelad i Program, ConsoleMenu och DogService med tydliga ansvarsområden. Belastningen i procent ska beräknas av en metod i DogService som ConsoleMenu anropar.

Kapacitet och belastning

Maxkapaciteten är **10 platser**. En liten hund tar **0,5 plats**, en medelstor hund tar **1 plats** och en stor hund tar **1,5 platser**. Belastningen beräknas som använd kapacitet delat med maxkapacitet, multiplicerat med 100.

De tre start hundarna använder tillsammans 3 platser. Då är 7 platser lediga och belastningen är 30 %. Systemet hanterar kapacitet, inte bokning av tider.

Betygskriterier från kursplanen

Examinationen bedömer främst din förmåga att felsöka och hantera fel strukturerat, använda Git, välja lämpliga datastrukturer, implementera grundläggande programlogik samt använda klasser, objekt och metoder med tydliga ansvarsområden.

Betygsnivåer

IG - Icke godkänt
Projektet kan inte byggas, centrala obligatoriska funktioner saknas eller fungerar inte, eller lösningen visar inte tillräcklig förmåga inom felsökning, felhantering och kodstruktur.

G - Godkänt
Samtliga obligatoriska funktioner fungerar. Kapacitetsberäkning, validering och felaktig inmatning hanteras stabilt. Lösningen använder tydlig programlogik, lämpliga datastrukturer och en begriplig objektorienterad struktur.

Instruktioner

1. Ladda ner startprojektet när examinationen börjar.
2. Skapa ett eget Git-repository och pusha startprojektet innan du börjar göra ändringar. 3. Bjud in läraren till repot.
4. Öppna lösningen och undersök vilka projekt, klasser och eventuella tester som finns.
5. Försök bygga och köra programmet innan du börjar ändra koden. Dokumentera de fel du upptäcker.
6. Prioritera först att projektet bygger och att de obligatoriska funktionerna fungerar. 7. Använd debugger, breakpoints, stegning och relevanta felmeddelanden vid felsökning. 8. Gör små och tydliga ändringar och committa regelbundet.
9. Testa samtliga funktioner och relevanta gränsfall. Kontrollera särskilt halvplatser, exakt full kapacitet, överfull kapacitet, utcheckning och felaktig inmatning.
10. Lämna in repot, inspelningen och din reflektion innan examinations tiden är slut.

Genomförande och inlämning

Tid: 12 timmar
Arbetsform: Individuellt arbete
Tillåtna verktyg: Git, Visual Studio eller Rider, debugger, terminal samt verktyg som ingår i startprojektet.
Generativ AI: Inte tillåtet.
Kommunikation: Ingen kommunikation eller koddelning med andra studenter under examinationen.

Du ska även lämna in en kort **REFLEKTION.md** eller **REFLEKTION.txt** där du beskriver vilka fel du identifierade, vilka ändringar du gjorde och hur du testade lösningen.